

Escuela Superior Politécnica del Litoral

Sistemas de Bases de Datos

Documentación del proyecto

Estudio de Impresión 3D

Grupo 1

Mora Eduardo

Santana James

Santiago Herrera

1. Triggers

Pusimos triggers para que la base haga sola dos cosas que antes había que acordarse de hacer a mano. Son dos tareas y tres triggers, porque el del subtotal se repite para cuando se inserta y para cuando se edita.

tr_detalle_subtotal

Cada vez que se agrega una línea a un pedido, este trigger calcula el subtotal multiplicando la cantidad por el precio unitario. Así el subtotal nunca queda mal escrito ni desactualizado, porque ya no depende de que la persona lo calcule.

```
DELIMITER $$
CREATE TRIGGER tr_detalle_subtotal
BEFORE INSERT ON DETALLE_PEDIDO
FOR EACH ROW
BEGIN
    SET NEW.Subtotal = NEW.Cantidad * NEW.Precio_Unitario;
END $$
DELIMITER ;
```

Es BEFORE INSERT porque el valor se tiene que calcular antes de que la fila se guarde. Al probarlo insertamos cantidad 4 y precio 22.50 dejando el subtotal en 0, y quedó guardado en 90.00. También hicimos el mismo trigger con BEFORE UPDATE (tr_detalle_subtotal_editar), para que el subtotal se recalcule si después se cambia la cantidad o el precio.

tr_consumo_descuenta_stock

Cuando se registra el material que gastó una orden de impresión, este trigger descuenta esa cantidad del inventario. Esto sale de una regla del negocio que ya teníamos escrita desde el avance anterior: al imprimir se descuenta el material usado.

```
DELIMITER $$
CREATE TRIGGER tr_consumo_descuenta_stock
AFTER INSERT ON CONSUMO_MATERIAL
FOR EACH ROW
BEGIN
    UPDATE MATERIAL
    SET Stock_Actual = Stock_Actual - (NEW.Material_Bueno + NEW.Material_Desperdiciado)
    WHERE ID_Material = NEW.ID_Material;
END $$
DELIMITER ;
```

Es AFTER INSERT porque primero se debe guardar el consumo y recién ahí tiene sentido tocar el inventario. Se descuenta el material bueno más el desperdiciado, porque los dos salieron de la bodega. En la prueba el stock pasó de 5000 a 4890 después de registrar un consumo de 100 más 10.

2. Reportes

Hicimos cuatro reportes como vistas. Cada uno junta tres o más tablas y muestra los valores descriptivos en vez de los códigos, o sea el nombre del cliente en lugar del número de cédula, el nombre del producto en lugar del código, y así.

Reporte	Tablas que junta	Para qué sirve
v_reporte_pedidos	PEDIDO, CLIENTE, DETALLE_PEDIDO, PRODUCTO	Ver qué pidió cada cliente, con el nombre del producto, la cantidad y el subtotal.
v_reporte_produccion	ORDEN_IMPRESION, PEDIDO, CLIENTE, IMPRESORA_3D, EMPLEADO	Saber en qué máquina y con qué operador se está imprimiendo cada pedido.
v_reporte_consumo	CONSUMO_MATERIAL, ORDEN_IMPRESION, MATERIAL, PEDIDO	Cuánto material se gastó por orden y cuánto costó, separando lo bueno de lo desperdiciado.
v_reporte_entregas	DESPACHO, PEDIDO, CLIENTE, TRANSPORTADORA, EMPLEADO	Quién envió cada pedido, con qué transportadora y en qué estado va.

Este es uno de ellos, para que se vea cómo se arma:

```
CREATE VIEW v_reporte_produccion AS
SELECT o.ID_Orden, o.Estado AS Estado_Orden,
       p.Numero_Pedido, c.Nombre AS Cliente,
       i.Marca, i.Modelo, i.Tecnologia,
       e.Nombre AS Operador,
       o.Gramos_Proyectados, o.Tiempo_Estimado
FROM ORDEN_IMPRESION o
JOIN PEDIDO p      ON o.Numero_Pedido = p.Numero_Pedido
JOIN CLIENTE c     ON p.Cedula_ID = c.Cedula_ID
JOIN IMPRESORA_3D i ON o.Codigo_Interno = i.Codigo_Interno
JOIN EMPLEADO e    ON o.ID_Empleado = e.ID_Empleado;
```

Las vistas también nos sirvieron para los permisos: hay usuarios que pueden ver el reporte pero no las tablas de donde salen los datos.

3. Procedimientos almacenados

Hicimos tres procedimientos por cada una de las 19 tablas del CRUD, o sea 57 en total: uno para insertar, uno para actualizar y uno para eliminar. Se llaman `sp_tabla_insertar`, `sp_tabla_actualizar` y `sp_tabla_eliminar`.

Todos siguen la misma estructura. Primero validan lo que se puede validar y avisan con SIGNAL si algo está mal, y después hacen el cambio dentro de una transacción con START TRANSACTION y COMMIT. Si algo falla en medio, el manejador de errores hace ROLLBACK para que la base no quede a medias.

```

DELIMITER $$
CREATE PROCEDURE sp_cliente_insertar(
    IN p_Cedula_ID varchar(13), IN p_Nombre varchar(100),
    IN p_Correo varchar(120), IN p_Telefono varchar(15))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '02000' SET MESSAGE_TEXT = 'No se pudo insertar en CLIENTE';
    END;

    IF EXISTS (SELECT 1 FROM CLIENTE WHERE Cedula_ID = p_Cedula_ID) THEN
        SIGNAL SQLSTATE '02000' SET MESSAGE_TEXT = 'Ya existe un registro con esa clave en
CLIENTE';
    END IF;

    START TRANSACTION;
    INSERT INTO CLIENTE (Cedula_ID, Nombre, Correo, Telefono)
    VALUES (p_Cedula_ID, p_Nombre, p_Correo, p_Telefono);
    COMMIT;
END $$
DELIMITER ;

```

Validaciones que pusimos en los procedimientos de insertar

- Al insertar, que la clave primaria no exista ya.
- Al actualizar o eliminar, que el registro exista; si no, avisa en vez de no hacer nada en silencio.
- En MATERIAL, que el stock no quede negativo (además la tabla tiene su CHECK, que protege también al actualizar).
- En PRODUCTO, que el precio y el stock no sean negativos.
- En DETALLE_PEDIDO, que la cantidad sea mayor que cero.
- En ENCUESTA_SATISFACCION, que las calificaciones estén entre 1 y 5 (la tabla también tiene el CHECK).

Probamos que el ROLLBACK sirve: intentamos insertar un pedido con un cliente que no existe y, después del error, la tabla seguía con las mismas 10 filas de antes.

4. Índices

Agregamos seis índices. La idea no fue ponerlos en todas las columnas, porque un índice ocupa espacio y hace más lentas las inserciones y actualizaciones. Los pusimos donde de verdad se busca seguido.

Índice	Tabla y columna	Por qué lo agregamos
idx_pedido_estado	PEDIDO(Estado_Actual)	El reporte de pedidos se filtra todo el tiempo por el estado, para ver los pendientes o los que están en impresión.
idx_producto_categoria	PRODUCTO(Categoria)	El catálogo se consulta y se ordena por categoría, que es la consulta que más se usa en la aplicación.

idx_cliente_nombre	CLIENTE(Nombre)	Al vendedor le sirve buscar al cliente por su nombre y no por la cédula, que casi nunca se la sabe de memoria.
idx_orden_fecha	ORDEN_IMPRESION(Fecha_Inicio)	Los reportes de producción se sacan por rango de fechas, por ejemplo lo del mes.
idx_encuesta_emp_fecha	ENCUESTA_SATISFACCION(ID_Empleado, Fecha_Respuesta)	Es un índice de dos columnas porque la consulta de calificaciones pide el empleado y un rango de fechas al mismo tiempo. Va primero el empleado, que se compara por igualdad, y después la fecha, que es un rango.
idx_consumo_material	CONSUMO_MATERIAL(ID_Material)	El reporte de consumo agrupa por material para saber cuánto se gastó de cada uno.

Las claves primarias ya tienen su índice creado solo, y las llaves foráneas también, así que esos no hizo falta agregarlos.

5. Usuarios y permisos

Creamos seis usuarios pensando en los roles que ya teníamos en el modelo. Cada uno tiene por lo menos dos permisos, y ninguno puede ver más de lo que necesita para su trabajo.

Usuario	Permisos	Tipo
admin_estudio	ALL PRIVILEGES sobre toda la base, con WITH GRANT OPTION	Base completa
vendedor01	SELECT, INSERT, UPDATE en CLIENTE SELECT, INSERT en PEDIDO SELECT en v_reporte_pedidos EXECUTE en sp_cliente_insertar	Tablas, vista y procedimiento
operador01	SELECT, UPDATE en ORDEN_IMPRESION SELECT, INSERT en CONSUMO_MATERIAL SELECT en v_reporte_produccion EXECUTE en sp_consumo_material_insertar	Tablas, vista y procedimiento
bodeguero01	SELECT, UPDATE en MATERIAL SELECT, INSERT en ENTRADA_MATERIAL SELECT en v_reporte_consumo	Tablas y vista
despachador01	SELECT, INSERT, UPDATE en DESPACHO SELECT en v_reporte_entregas EXECUTE en sp_despacho_actualizar	Tabla, vista y procedimiento
cliente_consulta	SELECT solo de las columnas Codigo_Producto, Nombre, Categoria y Precio de PRODUCTO SELECT en v_reporte_entregas	Columnas y vista

Con esto se cumplen los dos requisitos especiales: hay cinco permisos sobre vistas y tres permisos de ejecución sobre procedimientos.

Un caso que vale la pena mirar es cliente_consulta. No puede ver la tabla PRODUCTO completa, solo cuatro columnas, así que no se entera del stock ni de nada más. Y para saber

cómo va su envío usa la vista, sin tocar la tabla DESPACHO.

Así se creó uno de los usuarios:

```
CREATE USER 'operador01'@'localhost' IDENTIFIED BY 'Oper_2026';

GRANT SELECT, UPDATE ON estudio3d.ORDEN_IMPRESION TO 'operador01'@'localhost';
GRANT SELECT, INSERT ON estudio3d.CONSUMO_MATERIAL TO 'operador01'@'localhost';
GRANT SELECT ON estudio3d.v_reporte_produccion TO 'operador01'@'localhost';
GRANT EXECUTE ON PROCEDURE estudio3d.sp_consumo_material_insertar
  TO 'operador01'@'localhost';

FLUSH PRIVILEGES;
```

Para revisar los permisos de cualquiera se usa SHOW GRANTS FOR 'usuario'@'localhost'.

6. Cambio en la aplicación

La aplicación ya no manda INSERT, UPDATE ni DELETE directos. Ahora llama a los procedimientos, así que las validaciones y las transacciones quedan del lado de la base y no dependen del programa.

```
# antes
cursor.execute("INSERT INTO CLIENTE (Cedula_ID, Nombre, Correo, Telefono) "
               "VALUES (%s, %s, %s, %s)", (cedula_id, nombre, correo, telefono))

# ahora
cursor.execute("CALL sp_cliente_insertar(%s, %s, %s, %s)",
               (cedula_id, nombre, correo, telefono))
```

El mensaje que pone el SIGNAL le llega tal cual al usuario. Por ejemplo, si se repite una cédula, en pantalla sale "Error: Ya existe un registro con esa clave en CLIENTE".

7. Archivos que se entregan

Archivo	Qué contiene
schema_completo_estudio3d.sql	Todo junto: tablas, datos, triggers, reportes, procedimientos, índices y usuarios.
Diagrama_Schema_Estudio3D.pdf	El diagrama del esquema con las 19 tablas.
Documentacion_Proyecto_Grupo1.pdf	Este documento.
Manual_Usuario_Grupo1.pdf	El manual de la aplicación.
app/crud_estudio3d.py	El código de la aplicación.